Power BI Role Migration

How To Guide · v4 — All Four Actions

Full export/import + Selective export/import + Diff view + Dry run

Tabular Editor 2 · Macro Actions · DataWithSNS

What This Guide Covers

This guide covers four custom actions — two for export, two for import — plus first-time setup.

- **1.** Day-to-day workflow — right-click → Copy Roles → Export / Import
- **2.** One-time setup for new team members — installing the Macro Actions file

✓ If Macro Actions already installed, right-click shows 4 items under Copy Roles. New team member? Start with Appendix C. v3 users: replace MacroActions.json with the new v4 file.

Overview

Macro Actions add right-click menu items to Tabular Editor 2. Once installed, you copy roles between Power BI models in two clicks — no script editor, no copy/paste.

Item	Detail
What gets copied	All roles, members (Entra users + groups), RLS DAX filter expressions, ModelPermission
4 Macro Actions	Export All / Export Select / Import Standard / Import Select+Diff
What does NOT copy	OLS, measures, data, model structure
Where files live	C:\temp\PBIRoleCopy\ (auto-created on first export)
File format	roles_<ModelName>_<YYYYMMDD_HHMMSS>.json
Safety features	File picker, confirmation dialog, same-model check, audit log

Pre-Requisites

Item	Detail
Tabular Editor 2	Free — tabular.io/te2. AMO/TOM bundled — no extra installs
Macro Actions installed	MacroActions.json placed at %LOCALAPPDATA%\TabularEditor\ (see Appendix C)
Workspace tier	Premium, PPU, or Fabric (XMLA Read/Write required)
XMLA endpoint	Read Write enabled in Capacity Admin
Tenant setting	'Allow XMLA endpoints' = ON
Your access	Workspace Admin or Member (NOT Contributor) on both workspaces

⚠️ **XMLA Write must be enabled in your capacity admin portal. Without it, the import will fail silently or error on save.**

Part 1 — Export Roles from Prod Model

Two-click workflow once Macro Actions are installed.

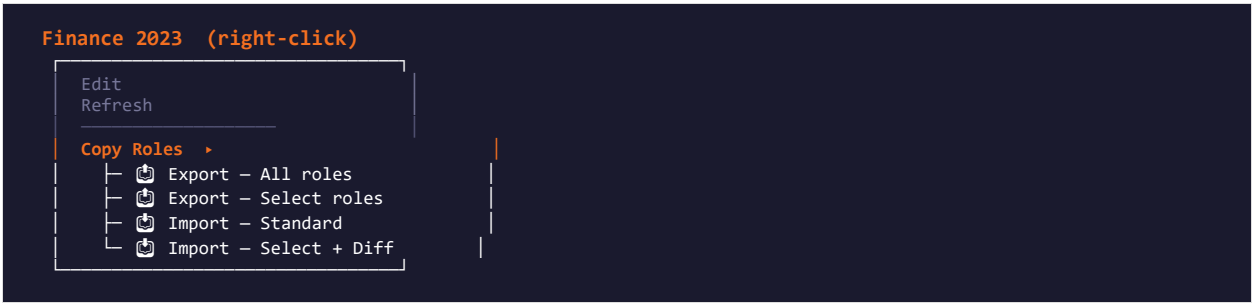
Steps

1	**Open Tabular Editor 2** Launch TE2 from Start menu
2	**Connect to Prod** File → Open → From DB
3	**Enter server** powerbi://api.powerbi.com/v1.0/myorg/YOUR_PROD_WORKSPACE → sign in (MFA)
4	**Select prod model** Pick the source semantic model → OK
5	**Right-click the model name** In the TOM Explorer tree (left side), right-click the model at the top
6	**Choose action** Click Copy Roles → 📤 Export — All roles
7	**Read confirmation popup** Shows: source model, role count, filename, save location
8	**Done** File written to C:\temp\PBIRoleCopy\roles_<ModelName>_<timestamp>.json

💡 Export only WRITES to disk. No save needed in TE2. No changes are made to the prod model itself.

What you see

Right-click menu in TE2:



Part 2 — Import Roles to Archive Model

Three-click workflow with built-in safety checks.

Steps

1	**Reconnect TE2 to Archive** File → Open → From DB (same TE2 window is fine)
2	**Enter archive server** powerbi://api.powerbi.com/v1.0/myorg/YOUR_ARCHIVE_WORKSPACE
3	**Select archive model** Pick the archive semantic model → OK
4	**Right-click the model name** In the TOM Explorer tree, right-click the model at the top
5	**Choose action** Click Copy Roles → Import — Standard
6	**File picker opens** Windows file picker shows C:\temp\PBIRoleCopy\ with the latest file pre-selected
7	**Select file → Open** Pick the export file. Filename includes source model + timestamp for easy identification
8	**Review confirmation dialog** Shows SOURCE → TARGET with role count + export date. VERIFY before clicking Yes
9	**Click Yes** Roles imported via ExecuteCommand — commits directly to Service
10	**DO NOT press Ctrl+S** ExecuteCommand has already committed. Saving causes errors

> ⚠️ **Do NOT press Ctrl+S after the import dialog closes. ExecuteCommand commits directly via XMLA. Pressing save triggers a conflict error.**

Confirmation Dialog Anatomy

Before any change is made to the archive model, this dialog appears. Read every line:

```
                    ── IMPORT CONFIRMATION ──

FILE:     roles__Finance_2023_20260514_1430.json
SOURCE:    Finance 2023
ROLES:    47
EXPORTED: 2026-05-14T14:30:21
BY:       sshetty7
                                          ↓ ↓ ↓
TARGET:    Finance Archive

WARNING: This will OVERWRITE existing roles with the same names.

Continue?  [Yes]  [No]
```

Verify before clicking Yes:

- FILE — should contain the source model name and a recent date
- SOURCE — must match the prod model you intended to copy from
- ROLES — count looks reasonable (not 0)
- EXPORTED — recent timestamp (or older if rolling back intentionally)
- BY — your username or a teammate's
- TARGET — must match the archive model you're currently connected to

Part 3 — Verify in Power BI Service

1	**Open Power BI Service** app.powerbi.com
2	**Open archive workspace** Find archive workspace in left panel
3	**Open semantic model settings** Click ··· next to archive model → Security
4	**Check roles list** All roles from prod should appear
5	**Check members on a role** Click a role → members listed below
6	**Test a role**

··· next to role → Test as role → verify RLS works

Folder Structure & Audit Log

All files live under one folder, auto-created on first export:

```
C:\temp\PBIRoleCopy\
├── audit.log                                    (timeline of all operations)
├── roles__Finance_2023_20260514_0930.json       (morning export)
├── roles__Finance_2023_20260514_1415.json       (afternoon export)
├── roles__Sales_20260513_1620.json              (different model, yesterday)
└── roles_HR_Dashboard_20260512_1045.json             (another model)
```

💡 Old exports stay forever. Useful for rollback — re-run Import and pick an older file. Delete manually when no longer needed.

Audit log format:

```
2026-05-14 09:30:21 | EXPORT |  Finance 2023 | 47 roles | roles__Finance_2023_20260514_0930.json
2026-05-14 09:32:15 | IMPORT |  Finance 2023 ->  Finance Archive | 47 roles |
roles__Finance_2023_20260514_0930.json
```

Troubleshooting

Item	Detail
Menu doesn't appear on right-click	See Appendix C — file might not be in correct location, or you didn't restart TE2
Folder not found error	Run Export first — folder auto-creates on first export
identityProvider 'default' error	Import script auto-fixes. If error persists, file may be corrupted — re-export
Members not visible in Service	Refresh Service page (F5). Check Security tab on archive model
Could not connect to server	Workspace name is case-sensitive. Verify XMLA Read/Write is ON
ExecuteCommand error on save	You pressed Ctrl+S. Reconnect to model and try again — don't save after import
RLS skipped warning	Archive model has different table names. DAX filters skip if table missing
Same Model Detected popup	You're importing into same model you exported from. Cancel unless intended
File picker shows no files	Folder is empty. Run Export first on a prod model

Appendix C — First-Time Macro Action Setup

Follow this once on each machine that doesn't already have Macro Actions installed. Takes about 30 seconds.

What you need

- Tabular Editor 2 already installed
- The MacroActions.json file (provided by team admin via SharePoint or email)

Setup steps

1	**Close Tabular Editor 2** If TE2 is open, close it completely before continuing
2	**Open File Explorer** Press Windows key + E
3	**Paste this path in address bar** Type or paste the path below into the address bar and press Enter

```
%LOCALAPPDATA%\TabularEditor
```

4	**Locate destination folder** Windows opens C:\Users\<your-username>\AppData\Local\TabularEditor
5	**Back up existing file (if any)** If MacroActions.json already exists, rename it to MacroActions_OLD.json before continuing
6	**Copy provided file here** Drag the MacroActions.json file (from SharePoint or email) into this folder
7	**Reopen Tabular Editor 2** Launch TE2 from Start menu
8	**Connect to any model** File → Open → From DB → connect to any Power BI model
9	**Verify menu appears** Right-click the model name in TOM Explorer → you should see 'Copy Roles' with two sub-items

Where the file goes (full path)

```
C:\Users\<username>\AppData\Local\TabularEditor\MacroActions.json
```

✓ Tip: The %LOCALAPPDATA% shortcut auto-resolves to your user folder. Just paste it in File Explorer address bar — Windows handles the rest.

If you have existing Macro Actions

If you already have a MacroActions.json with your own actions you want to keep:

- Open the existing file in a text editor (Notepad, VS Code)
- Open the provided MacroActions.json in another window
- The file is a JSON array — copy the two action objects from the provided file
- Paste them inside the [...] of your existing file, after the last comma
- Save and restart TE2

Verifying the install worked

After restarting TE2 and connecting to a model:

- Right-click the model name at the top of TOM Explorer
- You should see a Copy Roles submenu
- It should contain: Export to file and Import from file

⚠ **If the menu doesn't appear: confirm the file is at exactly %LOCALAPPDATA%\TabularEditor\MacroActions.json (not under Roaming or in a Documents folder). Restart TE2 fully. Try Tools menu → Macro Actions as a fallback.**

Sample MacroActions.json structure

For reference, the file is a JSON array containing two action objects:

```
[
  {
    "Name": "Copy Roles \\ Export to file",
    "Tooltip": "Export all roles + members + RLS to JSON",
    "Enabled": "true",
    "ValidContexts": "Model",
    "Execute": "using System.IO; ... (full export script)"
  },
  {
    "Name": "Copy Roles \\ Import from file",
    "Tooltip": "Pick a file and import roles",
    "Enabled": "true",
    "ValidContexts": "Model",
    "Execute": "using System.IO; ... (full import script)"
  }
]
```

Appendix A — Export Script (Reference)

This is the C# script embedded inside the 📤 Export to file custom action.

> 💡 You don't normally need to paste this manually. Use the right-click menu instead (see Part 1). This appendix exists for: maintenance, debugging, or if Macro Actions aren't installed yet.

```
using System.IO;

// ———————— config ————————
var folder      = @"C:\temp\PBIRoleCopy";
var sourceModel = Model.Database.Name;
var safeName    = System.Text.RegularExpressions.Regex.Replace(sourceModel, @"[^A-Za-z0-9]", "_");
var timestamp   = System.DateTime.Now.ToString("yyyyMMdd_HHmmss");
var fileName    = "roles_" + safeName + "_" + timestamp + ".json";
var filePath    = System.IO.Path.Combine(folder, fileName);

if (!Directory.Exists(folder)) Directory.CreateDirectory(folder);

// ———————— build TMSL operations ————————
var ops = new Newtonsoft.Json.Linq.JArray();

foreach (var role in Model.Roles)
{
    var members = new Newtonsoft.Json.Linq.JArray();
    foreach (var mem in role.Members)
    {
        members.Add(new Newtonsoft.Json.Linq.JObject(
            new Newtonsoft.Json.Linq.JProperty("memberName",       mem.MemberName),
            new Newtonsoft.Json.Linq.JProperty("memberId",         mem.MemberID),
            new Newtonsoft.Json.Linq.JProperty("identityProvider", "AzureAD")
        ));
    }

    var tps = new Newtonsoft.Json.Linq.JArray();
    foreach (var tp in role.TablePermissions)
    {
        if (string.IsNullOrEmpty(tp.FilterExpression)) continue;
        tps.Add(new Newtonsoft.Json.Linq.JObject(
            new Newtonsoft.Json.Linq.JProperty("name",             tp.Table.Name),
            new Newtonsoft.Json.Linq.JProperty("filterExpression", tp.FilterExpression)
        ));
    }

    ops.Add(new Newtonsoft.Json.Linq.JObject(
        new Newtonsoft.Json.Linq.JProperty("createOrReplace", new Newtonsoft.Json.Linq.JObject(
            new Newtonsoft.Json.Linq.JProperty("object", new Newtonsoft.Json.Linq.JObject(
                new Newtonsoft.Json.Linq.JProperty("database", "##TARGETDB##"),
                new Newtonsoft.Json.Linq.JProperty("role",     role.Name)
            )),
            new Newtonsoft.Json.Linq.JProperty("role", new Newtonsoft.Json.Linq.JObject(
                new Newtonsoft.Json.Linq.JProperty("name",            role.Name),
                new Newtonsoft.Json.Linq.JProperty("modelPermission", role.ModelPermission.ToString().ToLower()),
                new Newtonsoft.Json.Linq.JProperty("tablePermissions", tps),
                new Newtonsoft.Json.Linq.JProperty("members",         members)
            ))
        ))
    ));
}

// ———————— wrap with metadata envelope ————————
var envelope = new Newtonsoft.Json.Linq.JObject(
    new Newtonsoft.Json.Linq.JProperty("_meta", new Newtonsoft.Json.Linq.JObject(
        new Newtonsoft.Json.Linq.JProperty("sourceModel", sourceModel),
```

```
        new Newtonsoft.Json.Linq.JProperty("exportedAt",  System.DateTime.Now.ToString("o")),
        new Newtonsoft.Json.Linq.JProperty("exportedBy",  System.Environment.UserName),
        new Newtonsoft.Json.Linq.JProperty("roleCount",   Model.Roles.Count())
    )),
    new Newtonsoft.Json.Linq.JProperty("sequence", new Newtonsoft.Json.Linq.JObject(
        new Newtonsoft.Json.Linq.JProperty("operations", ops)
    ))
);

File.WriteAllText(filePath, envelope.ToString(Newtonsoft.Json.Formatting.Indented));

// ———————— audit log ————————
File.AppendAllText(System.IO.Path.Combine(folder, "audit.log"),
    System.DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss") + " | EXPORT | " +
    sourceModel + " | " + Model.Roles.Count() + " roles | " + fileName + "\r\n");

Info("Export complete!\n\n" +
     "Source:  " + sourceModel + "\n" +
     "Roles:   " + Model.Roles.Count() + "\n" +
     "File:    " + fileName + "\n\n" +
     "Saved to:\n" + folder);
```

Appendix B — Import Script (Reference)

This is the C# script embedded inside the Import from file custom action.

> 💡 You don't normally need to paste this manually. Use the right-click menu instead (see Part 2).

```
using System.IO;

var folder   = @"C:\temp\PBIRoleCopy";
var targetDb = Model.Database.Name;

if (!Directory.Exists(folder))
{
    Error("Folder not found: " + folder + "\n\nRun Export first.");
    return;
}

// file picker (default to latest)
var dlg = new System.Windows.Forms.OpenFileDialog();
dlg.InitialDirectory = folder;
dlg.Filter           = "Role export files|roles_*.json|All JSON files|*.json|All files|*.*";
dlg.Title            = "Select role export file to import into: " + targetDb;
dlg.RestoreDirectory = true;

var files = Directory.GetFiles(folder, "roles_*.json");
if (files.Length > 0)
{
    System.Array.Sort(files);
    dlg.FileName = System.IO.Path.GetFileName(files[files.Length - 1]);
}

if (dlg.ShowDialog() != System.Windows.Forms.DialogResult.OK) { Info("Cancelled."); return; }

var selectedFile = dlg.FileName;
```

```
var raw          = File.ReadAllText(selectedFile);
var json         = Newtonsoft.Json.Linq.JObject.Parse(raw);

// extract metadata
var sourceModel = "(unknown)"; var roleCount = "?";
var exportedAt  = "?";         var exportedBy = "?";
if (json["_meta"] != null) {
    if (json["_meta"]["sourceModel"] != null) sourceModel = json["_meta"]["sourceModel"].ToString();
    if (json["_meta"]["roleCount"]   != null) roleCount   = json["_meta"]["roleCount"].ToString();
    if (json["_meta"]["exportedAt"]  != null) exportedAt  = json["_meta"]["exportedAt"].ToString();
    if (json["_meta"]["exportedBy"]  != null) exportedBy  = json["_meta"]["exportedBy"].ToString();
}

// confirmation dialog
var msg = "——— IMPORT CONFIRMATION ———\n\n" +
          "FILE:     " + System.IO.Path.GetFileName(selectedFile) + "\n" +
          "SOURCE:   " + sourceModel + "\n" +
          "ROLES:    " + roleCount + "\n" +
          "EXPORTED: " + exportedAt + "\n" +
          "BY:       " + exportedBy + "\n\n        ↓ ↓ ↓\n\n" +
          "TARGET:   " + targetDb + "\n\n" +
          "WARNING: This will OVERWRITE existing roles.\n\nContinue?";

var ok = System.Windows.Forms.MessageBox.Show(msg, "Confirm Role Import",
    System.Windows.Forms.MessageBoxButtons.YesNo,
    System.Windows.Forms.MessageBoxIcon.Warning);
if (ok != System.Windows.Forms.DialogResult.Yes) { Info("Cancelled."); return; }

// same-model safety check
if (sourceModel == targetDb) {
    var ok2 = System.Windows.Forms.MessageBox.Show(
        "Source and target are the SAME model:\n\n" + targetDb + "\n\nReally continue?",
        "Same Model Detected",
        System.Windows.Forms.MessageBoxButtons.YesNo,
        System.Windows.Forms.MessageBoxIcon.Stop);
    if (ok2 != System.Windows.Forms.DialogResult.Yes) { Info("Cancelled."); return; }
}

// transform + execute
var tmsl = raw.Replace("##TARGETDB##", targetDb).Replace("\"default\"", "\"AzureAD\"");
var parsed = Newtonsoft.Json.Linq.JObject.Parse(tmsl);
if (parsed["_meta"] != null) parsed.Remove("_meta");
tmsl = parsed.ToString(Newtonsoft.Json.Formatting.None);

ExecuteCommand(tmsl);

// audit log
File.AppendAllText(System.IO.Path.Combine(folder, "audit.log"),
    System.DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss") + " | IMPORT | " +
    sourceModel + " -> " + targetDb + " | " + roleCount + " roles | " +
    System.IO.Path.GetFileName(selectedFile) + "\r\n");

Info("Import complete!\n\n" + sourceModel + "\n  ->  " + targetDb +
     "\n\n" + roleCount + " roles imported.");
```

Part 3 — Export Selected Roles

Use when new roles created locally need pushing to target models. Lets you pick only new roles.

1	**Connect TE2 to source model** File → Open → From DB → source/golden model
2	**Right-click model name** Copy Roles → 📥 Export — Select roles
3	**Checkbox UI opens** Shows all roles in the model with filter and toggle buttons
4	**Click ★ New button** Auto-ticks roles NOT in the last export file (new roles only)
5	**Or filter by name** Type market code in filter box (e.g. '9901') to narrow the list
6	**Click Export Selected** File saved as roles_<Model>_partial_<timestamp>.json

💡 ★ New compares against the latest export file in C:\temp\PBIRoleCopy\. On first-ever run, no previous file exists — use the filter box to manually select new roles.

Part 4 — Import Select + Diff

Use when you want to see exactly what will change before committing, or import only specific roles.

1	**Connect TE2 to target model** File → Open → From DB → target model
2	**Right-click model name** Copy Roles → 📥 Import — Select + Diff
3	**File picker opens** Pick the export file (partial or full)
4	**Diff view loads** List shows: [NEW] in green, [UPDATE] in orange based on current model roles
5	**Click ● New only** Ticks only new roles — skip existing role updates
6	**Optional: Dry Run** Check Dry Run → click Import → shows summary with no changes made
7	**Click Import Selected** Confirmation dialog shows new count + update count → Yes to commit

⚠ **Do NOT press Ctrl+S after import. ExecuteCommand already committed via XMLA.**

New Role Workflow

Complete workflow for pushing new roles to 5 target models:

1	**Create new roles in source model** Add roles locally in TE2 with correct DAX filter expressions
2	**Export — Select roles** Right-click → 📤 Export — Select roles → ★ New → Export Selected
3	**Connect to model 1** File → Open → From DB → first target workspace
4	**Import to model 1** Right-click → 📥 Import — Standard → pick partial file → confirm
5	**Repeat for models 2–5** Reconnect TE2 for each target → Import — same file each time
6	**Assign members in Service** app.powerbi.com → each model → Security → new roles → add members

💡 The ##TARGETDB## placeholder in the export file gets replaced automatically with the name of whatever model TE2 is connected to at import time. One file works for all 5 models.

Appendix D — Selective Export Script Key Notes

💡 You don't need to paste this manually. Use right-click menu. Provided for maintenance.

```
#r "System.Drawing"  // required — adds assembly reference in TE2
using System.Drawing;   // enables Color, Font, FontStyle without full qualification

// UI: CheckedListBox of all roles in current model
// ★ New: reads latest roles_*.json, auto-ticks roles not in that file
// Partial export: saved as roles_<Model>_partial_<timestamp>.json
// _meta.exportType = 'partial' | 'full'  (readable in confirmation dialog)
```

Appendix E — Selective Import Script Key Notes

💡 You don't need to paste this manually. Use right-click menu. Provided for maintenance.

```
#r "System.Drawing"  // required — adds assembly reference in TE2
using System.Drawing;   // enables Color, Font, SolidBrush, Rectangle
```

```
// Diff: compares file role names vs Model.Roles → NEW vs UPDATE
// Owner-drawn CheckedListBox: green rows = NEW, orange rows = UPDATE
// ● New only / ● Update only buttons for quick filtering
// Dry Run: shows what WOULD happen, no ExecuteCommand called
// Subset TMSL: rebuilds operations array with only selected roles
// Audit log entry: 'N roles (X new, Y upd)'
```

Quick Reference Card

Item	Detail
Folder	C:\temp\PBIRoleCopy\
Macro Actions file	%LOCALAPPDATA%\TabularEditor\MacroActions.json
Audit log	C:\temp\PBIRoleCopy\audit.log
File naming	roles_<ModelName>_<YYYYMMDD_HHMMSS>.json
XMLA format	powerbi://api.powerbi.com/v1.0/myorg/WORKSPACE_NAME
Export workflow	Right-click model → Copy Roles → Export — All roles
Import workflow	Right-click model → Copy Roles → Import — Standard
After export?	Confirmation popup — no save needed
After import?	Confirmation popup — DO NOT press Ctrl+S
Verify result	Service → Workspace → Model → ··· → Security
Rollback	Re-run Import action → pick older file → confirm